Python Programming Fundamentals

List Basics

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. What is a List?

A **list** is an ordered, mutable collection of items.

```
fruits = ["apple", "banana", "cherry"]
numbers = [10, 20, 30, 40, 50]
mixed = [42, "hello", 3.14, True]
empty = []
nested = [[1, 2], [3, 4], [5, 6]]
```

1. What is a List?

A **list** is an ordered, mutable collection of items.

```
fruits = ["apple", "banana", "cherry"]
numbers = [10, 20, 30, 40, 50]
mixed = [42, "hello", 3.14, True]
empty = []
nested = [[1, 2], [3, 4], [5, 6]]
```

Key properties:

- **Ordered** — items have a fixed position (index)
- **Mutable** — you can add, remove, or change items
- **Allow duplicates** — the same value can appear multiple times
- **Mixed types** — a single list can hold different types (but usually stick to one type)

Outline

2. Indexing

Each item in a list has an **index** — its position.

```
fruits = ["apple", "banana", "cherry", "date"]
#           0         1         2        3      (positive)
#          -4        -3        -2       -1      (negative)

print(fruits[0])    # apple
print(fruits[2])    # cherry
print(fruits[-1])   # date    (last item)
print(fruits[-2])   # cherry  (second to last)
```

2. Indexing

Each item in a list has an **index** — its position.

```
fruits = ["apple", "banana", "cherry", "date"]
#           0         1         2        3      (positive)
#          -4        -3        -2       -1      (negative)

print(fruits[0])    # apple
print(fruits[2])    # cherry
print(fruits[-1])   # date    (last item)
print(fruits[-2])   # cherry  (second to last)
```

Modifying an item by index:

```
fruits[1] = "blueberry"
print(fruits)   # ['apple', 'blueberry', 'cherry', 'date']
```

Outline

3. Slicing

Slicing extracts a **sub-list** using `[start:stop:step]`.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5])     # [2, 3, 4]   stop is exclusive
print(numbers[:4])      # [0, 1, 2, 3]
print(numbers[6:])      # [6, 7, 8, 9]
print(numbers[::2])     # [0, 2, 4, 6, 8]  every 2nd element
print(numbers[::-1])    # [9, 8, ..., 0]   reversed
```

3. Slicing

Slicing extracts a **sub-list** using `[start:stop:step]`.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5])     # [2, 3, 4]   stop is exclusive
print(numbers[:4])      # [0, 1, 2, 3]
print(numbers[6:])      # [6, 7, 8, 9]
print(numbers[::2])     # [0, 2, 4, 6, 8]  every 2nd element
print(numbers[::-1])    # [9, 8, ..., 0]   reversed
```

Practical uses:

```
data = [10, 20, 30, 40, 50, 60, 70]
first_three = data[:3]    # [10, 20, 30]
last_two    = data[-2:]   # [60, 70]
middle      = data[2:-2]  # [30, 40, 50]
copy        = data[:]     # full copy
```

Outline

4. len() and Membership

```
fruits = ["apple", "banana", "cherry"]

print(len(fruits))                  # 3
print("banana" in fruits)           # True
print("grape" in fruits)            # False
print("grape" not in fruits)        # True
```

4. len() and Membership

```
fruits = ["apple", "banana", "cherry"]

print(len(fruits))                  # 3
print("banana" in fruits)           # True
print("grape" in fruits)            # False
print("grape" not in fruits)        # True

# Safe access — avoid IndexError
def safe_get(lst, index):
    if 0 <= index < len(lst):
        return lst[index]
    return None

scores = [85, 92, 78]
print(safe_get(scores, 1))    # 92
print(safe_get(scores, 10))   # None
```

Outline

5. Nested Lists

```
student = ["Alice", 21, "Dublin", [85, 92, 78]]
name    = student[0]      # "Alice"
scores = student[3]      # [85, 92, 78]

# Access nested items
first_score = student[3][0]    # 85
```

5. Nested Lists

5. Nested Lists

```
student = ["Alice", 21, "Dublin", [85, 92, 78]]
name    = student[0]      # "Alice"
scores = student[3]      # [85, 92, 78]

# Access nested items
first_score = student[3][0]    # 85
```

A 2D grid as a list of lists:

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]

print(grid[1][2])   # 6  (row 1, column 2)
```

5. Nested Lists

```
for row in grid:
    print(" ".join(str(cell) for cell in row))
```

Outline

6. Building Lists Dynamically

```
# Start empty, add items one by one
squares = []
for i in range(1, 8):
    squares.append(i ** 2)
print(squares)   # [1, 4, 9, 16, 25, 36, 49]
```

6. Building Lists Dynamically

6. Building Lists Dynamically

```
# Start empty, add items one by one
squares = []
for i in range(1, 8):
    squares.append(i ** 2)
print(squares)   # [1, 4, 9, 16, 25, 36, 49]

# Collecting user input into a list
names = []
print("Enter names (blank line to stop):")
while True:
    name = input("> ").strip()
    if not name:
        break
    names.append(name)

print(f"You entered {len(names)} names:")
```

6. Building Lists Dynamically

```
for i, name in enumerate(names, start=1):
    print(f"  {i}. {name}")
```

Outline

7. Converting Between Types

```
# str -> list -> str
sentence = "the quick brown fox"
words = sentence.split()
rejoined = " ".join(words)
print(words)      # ['the', 'quick', 'brown', 'fox']
print(rejoined)   # the quick brown fox

# range -> list
nums = list(range(1, 6))
print(nums)    # [1, 2, 3, 4, 5]

# string -> list of characters
chars = list("Python")
print(chars)   # ['P', 'y', 't', 'h', 'o', 'n']
```

Thanks for Watching - Any questions?